

Waypoints

Name your subslides, not count them.

A Touying Waypoints Guide

The Problem

Why waypoints?

With numeric indices, inserting a #pause shifts every number:

Before — correct:

```
A #pause B #pause C
#uncover("2-")[From 2]
#only(3)[Only on 3]
```

After adding a pause — broken:

```
A #pause A2 #pause B #pause C
#uncover("2-")[From 2] // wrong!
#only(3)[Only on 3]    // wrong!
```

Why waypoints?

With numeric indices, inserting a `#pause` shifts every number:

Before — correct:

```
A #pause B #pause C
#uncover("2-")[From 2]
#only(3)[Only on 3]
```

After adding a pause — broken:

```
A #pause A2 #pause B #pause C
#uncover("2-")[From 2] // wrong!
#only(3)[Only on 3]    // wrong!
```

Waypoints replace fragile numbers with stable names:

```
#waypoint(<reveal>)           // named pause
#uncover(<reveal>)[content]    // refers to the name
```

And in reality we don't need to know every single animation step, special ones are often enough. Note: Waypoints work in contexts like `fletcher`, but not inside text blocks like `raw`.

Explicit Waypoints

#waypoint + #uncover

Content before.

```
Content before.  
#waypoint(<demo>)  
First Content.  
#pause  
Second Content.  
#uncover(<demo>)[  
  Revealed during waypoint.  
]
```

#waypoint acts like #pause but names the position. #uncover(<lbl>) shows content during that waypoint's range.

#waypoint + #uncover

```
Content before.  
#waypoint(<demo>)  
First Content.  
#pause  
Second Content.  
#uncover(<demo>)[  
  Revealed during waypoint.  
]
```

#waypoint acts like #pause but names the position. #uncover(<lbl>) shows content during that waypoint's range.

Content before. First Content. Revealed during waypoint.

#waypoint + #uncover

```
Content before.  
#waypoint(<demo>)  
First Content.  
#pause  
Second Content.  
#uncover(<demo>)[  
  Revealed during waypoint.  
]
```

#waypoint acts like #pause but names the position. #uncover(<lbl>) shows content during that waypoint's range.

Content before. First Content. Second Content. Revealed during waypoint.

#waypoint + #uncover (multiple)

```
#waypoint(<intro>)  
Intro phase.  
#pause  
More Intro.  
#waypoint(<detail>)  
Detail phase.  
#uncover(<intro>)[During intro.]  
#uncover(<detail>)[During detail.]
```

Each waypoint owns a range of subslides.
Content uncovered with a label is visible
only during that range.

#waypoint + #uncover (multiple)

Intro phase. During *intro*.

```
#waypoint(<intro>)  
Intro phase.  
#pause  
More Intro.  
#waypoint(<detail>)  
Detail phase.  
#uncover(<intro>)[During intro.]  
#uncover(<detail>)[During detail.]
```

Each waypoint owns a range of subslides.
Content uncovered with a label is visible
only during that range.

#waypoint + #uncover (multiple)

Intro phase. More Intro. During *intro*.

```
#waypoint(<intro>)  
Intro phase.  
#pause  
More Intro.  
#waypoint(<detail>)  
Detail phase.  
#uncover(<intro>)[During intro.]  
#uncover(<detail>)[During detail.]
```

Each waypoint owns a range of subslides.
Content uncovered with a label is visible
only during that range.

#waypoint + #uncover (multiple)

```
#waypoint(<intro>)  
Intro phase.  
#pause  
More Intro.  
#waypoint(<detail>)  
Detail phase.  
#uncover(<intro>)[During intro.]  
#uncover(<detail>)[During detail.]
```

Intro phase. More Intro. Detail phase.
During *detail*.

Each waypoint owns a range of subslides.
Content uncovered with a label is visible
only during that range.

#waypoint + #effect

Normal text. Red during <hl>.

```
Normal text.  
#waypoint(<hl>)  
#effect(  
  text.with(fill: red), <hl>  
) [Red during <hl>.]
```

#effect(fn, <lbl>) applies a transform while the waypoint is active.

#waypoint + #effect

Normal text. Red during <hl>.

```
Normal text.  
#waypoint(<hl>)  
#effect(  
  text.with(fill: red), <hl>  
) [Red during <hl>.]
```

#effect(fn, <lbl>) applies a transform while the waypoint is active.

No-advance waypoint

Still subslide 1. Visible immediately.

```
#waypoint(<here>, advance: false)
Still subslide 1.
#uncover(<here>)[
  Visible immediately.
]
```

advance: false marks the position **without** creating a new subslide.

Explicit Start Index

```
#waypoint(<end>, start: 3)
Ends at subslide 3.
#v(4em)
#waypoint(<start>, start:1)
We start down here.
#pause
And will continue further up.
```

We start down here.

`start: int` starts the waypoint at a specific subslide index. `start: <lbl>` starts it at another waypoint's position.

Explicit Start Index

```
#waypoint(<end>, start: 3)
Ends at subslide 3.
#v(4em)
#waypoint(<start>, start:1)
We start down here.
#pause
And will continue further up.
```

start: int starts the waypoint at a specific subslide index. start: <lbl> starts it at another waypoint's position.

We start down here.
And will continue further up.

Explicit Start Index

Ends at subslide 3.

```
#waypoint(<end>, start: 3)
Ends at subslide 3.
#v(4em)
#waypoint(<start>, start:1)
We start down here.
#pause
And will continue further up.
```

We start down here.

And will continue further up.

`start: int` starts the waypoint at a specific subslide index. `start: <lbl>` starts it at another waypoint's position.

Querying Waypoints

get-first / get-last

```
#waypoint(<p>)  
Start. #pause  
Continued.  
#waypoint(<q>)  
Next.  
#only(get-first(<p>))[First of p.]  
#only(get-last(<p>))[Last of p.]
```

A waypoint spanning multiple subslides (due to #pause inside its range) can be queried at its edges.

get-first / get-last

Start.

First of <p> only.

```
#waypoint(<p>)  
Start. #pause  
Continued.  
#waypoint(<q>)  
Next.  
#only(get-first(<p>))[First of p.]  
#only(get-last(<p>))[Last of p.]
```

A waypoint spanning multiple subslides (due to #pause inside its range) can be queried at its edges.

get-first / get-last

Start. Continued.

Last of <p> only.

```
#waypoint(<p>)  
Start. #pause  
Continued.  
#waypoint(<q>)  
Next.  
#only(get-first(<p>))[First of p.]  
#only(get-last(<p>))[Last of p.]
```

A waypoint spanning multiple subslides (due to #pause inside its range) can be queried at its edges.

get-first / get-last

Start. Continued. Next.

```
#waypoint(<p>)  
Start. #pause  
Continued.  
#waypoint(<q>)  
Next.  
#only(get-first(<p>))[First of p.]  
#only(get-last(<p>))[Last of p.]
```

A waypoint spanning multiple subslides (due to #pause inside its range) can be queried at its edges.

from-wp — onward from a waypoint

```
#waypoint(<step>)  
Step content.  
#waypoint(<later>)  
Later content.\br/>#uncover(from-wp(<step>))[  
  From step onward – through `<later>` too.  
]
```

from-wp(<lbl>) is visible from the waypoint's first subslide to the **end of the slide**. Unlike a bare label, it is not bounded by the next waypoint.

from-wp — onward from a waypoint

```
#waypoint(<step>)  
Step content.  
#waypoint(<later>)  
Later content.\br/>#uncover(from-wp(<step>))[  
  From step onward — through `<later>` too.  
]
```

from-wp(<lbl>) is visible from the waypoint's first subslide to the **end of the slide**. Unlike a bare label, it is not bounded by the next waypoint.

Step content.

From <step> onward — through <later> too.

from-wp — onward from a waypoint

```
#waypoint(<step>)  
Step content.  
#waypoint(<later>)  
Later content.\n#uncover(from-wp(<step>))[  
  From step onward — through `<later>` too.  
]
```

from-wp(<lbl>) is visible from the waypoint's first subslide to the **end of the slide**. Unlike a bare label, it is not bounded by the next waypoint.

Step content. Later content.

From <step> onward — through <later> too.

until-wp — before a waypoint

Before

phase 2.

```
#waypoint(<phase-1>)  
Phase 1 content.  
#waypoint(<phase-2>)  
Phase 2 content.  
#uncover(until-wp(<phase-2>))[  
  Before phase 2.  
]  
#uncover(from-wp(<phase-2>))[  
  Only from phase 2.  
]
```

until-wp(<lbl>) is visible on all subslides **before** the waypoint starts — including subslides before any waypoint is reached.

until-wp — before a waypoint

```
#waypoint(<phase-1>)  
Phase 1 content.  
#waypoint(<phase-2>)  
Phase 2 content.  
#uncover(until-wp(<phase-2>)) [  
  Before phase 2.  
]  
#uncover(from-wp(<phase-2>)) [  
  Only from phase 2.  
]
```

Phase 1 content.
phase 2.

Before

until-wp(<lbl>) is visible on all subslides **before** the waypoint starts — including subslides before any waypoint is reached.

until-wp — before a waypoint

```
#waypoint(<phase-1>)  
Phase 1 content.  
#waypoint(<phase-2>)  
Phase 2 content.  
#uncover(until-wp(<phase-2>))[  
  Before phase 2.  
]  
#uncover(from-wp(<phase-2>))[  
  Only from phase 2.  
]
```

`until-wp(<lbl>)` is visible on all subslides **before** the waypoint starts — including subslides before any waypoint is reached.

Phase 1 content. Phase 2 content.
Only from phase 2.

Inverted waypoints

```
Base Content
#waypoint(<middle>)
Middle - 1
#pause
Middle - 2
#waypoint(<end>)
End Content

#alternatives(at: (not-wp(<middle>), <middle>)
)[Not during middle.][During middle.]
```

Similar to “!” for string ranges, we can use `<not-wp>` to invert the selection based on a waypoint.

Base Content

Not during middle.

Inverted waypoints

```
Base Content
#waypoint(<middle>)
Middle - 1
#pause
Middle - 2
#waypoint(<end>)
End Content

#alternatives(at: (not-wp(<middle>), <middle>)
)[Not during middle.][During middle.]
```

Similar to “!” for string ranges, we can use `<not-wp>` to invert the selection based on a waypoint.

Base Content

Middle - 1

During middle.

Inverted waypoints

```
Base Content
#waypoint(<middle>)
Middle - 1
#pause
Middle - 2
#waypoint(<end>)
End Content

#alternatives(at: (not-wp(<middle>), <middle>)
)[Not during middle.][During middle.]
```

Similar to “!” for string ranges, we can use `<not-wp>` to invert the selection based on a waypoint.

Base Content

Middle - 1

Middle - 2

During middle.

Inverted waypoints

```
Base Content
#waypoint(<middle>)
Middle - 1
#pause
Middle - 2
#waypoint(<end>)
End Content

#alternatives(at: (not-wp(<middle>), <middle>)
)[Not during middle.][During middle.]
```

Similar to “!” for string ranges, we can use `<not-wp>` to invert the selection based on a waypoint.

Base Content

Middle - 1

Middle - 2

End Content

Not during middle.

Ranges & Navigation

Bounded range

```
#waypoint(<ra>)  
Range A.  
#waypoint(<rb>)  
Range B.  
#waypoint(<rc>)  
Range C.  
#uncover(  
    (from-wp(<ra>), until-wp(<rc>))  
)[During A and B.]
```

Combine from-wp and until-wp in an array to span a range. Runs from <ra> up to (but not including) <rc>.

Bounded range

Range A.

During A and B.

```
#waypoint(<ra>)  
Range A.  
#waypoint(<rb>)  
Range B.  
#waypoint(<rc>)  
Range C.  
#uncover(  
    (from-wp(<ra>), until-wp(<rc>))  
)[During A and B.]
```

Combine from-wp and until-wp in an array to span a range. Runs from <ra> up to (but not including) <rc>.

Bounded range

Range A. Range B.

During A and B.

```
#waypoint(<ra>)  
Range A.  
#waypoint(<rb>)  
Range B.  
#waypoint(<rc>)  
Range C.  
#uncover(  
    (from-wp(<ra>), until-wp(<rc>))  
)[During A and B.]
```

Combine from-wp and until-wp in an array to span a range. Runs from <ra> up to (but not including) <rc>.

Bounded range

Range A. Range B. Range C.

```
#waypoint(<ra>)  
Range A.  
#waypoint(<rb>)  
Range B.  
#waypoint(<rc>)  
Range C.  
#uncover(  
    (from-wp(<ra>), until-wp(<rc>))  
)[During A and B.]
```

Combine from-wp and until-wp in an array to span a range. Runs from <ra> up to (but not including) <rc>.

prev-wp / next-wp

```
#waypoint(<na>)
Section A.
#waypoint(<nb>)
Section B.
#waypoint(<nc>)
Section C.
#only(next-wp(<na>))[
    During B (next after A).
]
#only(prev-wp(<nc>))[
    During B (prev before C).
]
```

Jump to the adjacent waypoint in subslide order. In fact you may also pass an amount to jump multiple waypoints: `next-wp(<lbl>, amount: 2)`.

prev-wp / next-wp

Section A.

```
#waypoint(<na>)  
Section A.  
#waypoint(<nb>)  
Section B.  
#waypoint(<nc>)  
Section C.  
#only(next-wp(<na>))[  
    During B (next after A).  
]  
#only(prev-wp(<nc>))[  
    During B (prev before C).  
]
```

Jump to the adjacent waypoint in subslide order. In fact you may also pass an amount to jump multiple waypoints: `next-wp(<lbl>, amount: 2)`.

prev-wp / next-wp

```
#waypoint(<na>)
Section A.
#waypoint(<nb>)
Section B.
#waypoint(<nc>)
Section C.
#only(next-wp(<na>))[
    During B (next after A).
]
#only(prev-wp(<nc>))[
    During B (prev before C).
]
```

Section A. Section B. During B
(next after A). During B (prev before C).

Jump to the adjacent waypoint in subslide order. In fact you may also pass an amount to jump multiple waypoints: `next-wp(<lbl>, amount: 2)`.

prev-wp / next-wp

```
#waypoint(<na>)  
Section A.  
#waypoint(<nb>)  
Section B.  
#waypoint(<nc>)  
Section C.  
#only(next-wp(<na>))[  
    During B (next after A).  
]  
#only(prev-wp(<nc>))[  
    During B (prev before C).  
]
```

Section A. Section B. Section C.

Jump to the adjacent waypoint in subslide order. In fact you may also pass an amount to jump multiple waypoints: `next-wp(<lbl>, amount: 2)`.

Composing shifts with `from-wp/until-wp`

Only

during A.

```
#waypoint(<ca>)  
Part A.  
#waypoint(<cb>)  
Part B.  
#waypoint(<cc>)  
Part C.  
// from-wp(next-wp(<ca>)) = from B  
#uncover(from-wp(next-wp(<ca>)))[  
  From B onward.  
]  
// until-wp(prev-wp(<cc>)) = until B  
#uncover(until-wp(prev-wp(<cc>)))[  
  Only during A.  
]
```

Shifts compose naturally with `from-wp/`
`until-wp`.

Composing shifts with `from-wp/until-wp`

```
#waypoint(<ca>)  
Part A.  
#waypoint(<cb>)  
Part B.  
#waypoint(<cc>)  
Part C.  
// from-wp(next-wp(<ca>)) = from B  
#uncover(from-wp(next-wp(<ca>)))[  
  From B onward.  
]  
// until-wp(prev-wp(<cc>)) = until B  
#uncover(until-wp(prev-wp(<cc>)))[  
  Only during A.  
]
```

Part A.
during A.

Only

Shifts compose naturally with `from-wp/`
`until-wp`.

Composing shifts with `from-wp/until-wp`

Part A. Part B.

From B onward.

```
#waypoint(<ca>)
Part A.
#waypoint(<cb>)
Part B.
#waypoint(<cc>)
Part C.
// from-wp(next-wp(<ca>)) = from B
#uncover(from-wp(next-wp(<ca>)))[
  From B onward.
]
// until-wp(prev-wp(<cc>)) = until B
#uncover(until-wp(prev-wp(<cc>)))[
  Only during A.
]
```

Shifts compose naturally with `from-wp/`
`until-wp`.

Composing shifts with `from-wp/until-wp`

Part A. Part B. Part C. From B onward.

```
#waypoint(<ca>)
Part A.
#waypoint(<cb>)
Part B.
#waypoint(<cc>)
Part C.
// from-wp(next-wp(<ca>)) = from B
#uncover(from-wp(next-wp(<ca>)))[
  From B onward.
]
// until-wp(prev-wp(<cc>)) = until B
#uncover(until-wp(prev-wp(<cc>)))[
  Only during A.
]
```

Shifts compose naturally with `from-wp/`
`until-wp`.

Inclusive range via next-wp

```
#waypoint(<ia>)  
Part A.  
#waypoint(<ib>)  
Part B.  
#waypoint(<ic>)  
Part C.  
// A only (exclusive of B)  
#only((from-wp(<ia>), until-wp(<ib>)))[  
    Exactly A.  
]  
// A and B (inclusive)  
#only((from-wp(<ia>),  
    next-wp(until-wp(<ib>)))  
)[A and B.]
```

`next-wp(until-wp(<ib>))` → `until-wp(<ic>)`, so `<ib>` is included.

Inclusive range via next-wp

```
#waypoint(<ia>)
Part A.
#waypoint(<ib>)
Part B.
#waypoint(<ic>)
Part C.
// A only (exclusive of B)
#only((from-wp(<ia>), until-wp(<ib>)))[
  Exactly A.
]
// A and B (inclusive)
#only((from-wp(<ia>),
  next-wp(until-wp(<ib>)))
)[A and B.]
```

`next-wp(until-wp(<ib>))` → `until-wp(<ic>)`, so `<ib>` is included.

Part A. Exactly during A.
During A and B (inclusive).

Inclusive range via next-wp

```
#waypoint(<ia>)  
Part A.  
#waypoint(<ib>)  
Part B.  
#waypoint(<ic>)  
Part C.  
// A only (exclusive of B)  
#only((from-wp(<ia>), until-wp(<ib>)))[  
  Exactly A.  
]  
// A and B (inclusive)  
#only((from-wp(<ia>),  
  next-wp(until-wp(<ib>)))  
)[A and B.]
```

`next-wp(until-wp(<ib>))` → `until-wp(<ic>)`, so `<ib>` is included.

Part A. Part B.
(inclusive).

During A and B

Inclusive range via next-wp

Part A. Part B. Part C.

```
#waypoint(<ia>)  
Part A.  
#waypoint(<ib>)  
Part B.  
#waypoint(<ic>)  
Part C.  
// A only (exclusive of B)  
#only((from-wp(<ia>), until-wp(<ib>)))[  
  Exactly A.  
]  
// A and B (inclusive)  
#only((from-wp(<ia>),  
  next-wp(until-wp(<ib>)))  
)[A and B.]
```

`next-wp(until-wp(<ib>))` → `until-wp(<ic>)`, so `<ib>` is included.

Implicit Waypoints

Labels as auto-waypoints

Always visible.

```
Always visible.  
#uncover(<imp-rev>)[  
  Appears via implicit wp.  
]
```

Using a label in `#uncover`, `#only`, or `#effect` creates a waypoint automatically — no `#waypoint` call needed.

Labels as auto-waypoints

```
Always visible.  
#uncover(<imp-rev>)[  
  Appears via implicit wp.  
]
```

Using a label in `#uncover`, `#only`, or `#effect` creates a waypoint automatically — no `#waypoint` call needed.

Always visible. Appears via implicit waypoint.

Multiple implicit

Base content.

Phase B styled.

Base content.

```
#uncover(<im-a>)[Phase A.]  
#effect(  
  text.with(fill: blue), <im-b>  
)[Phase B styled.]  
#only(<im-c>)[Phase C only.]
```

Each distinct label → one implicit waypoint.

Reusing a label adds no extra subslides.

Multiple implicit

Base content. Phase A. Phase B styled.

```
Base content.  
#uncover(<im-a>)[Phase A.]  
#effect(  
  text.with(fill: blue), <im-b>  
)[Phase B styled.]  
#only(<im-c>)[Phase C only.]
```

Each distinct label → one implicit waypoint.
Reusing a label adds no extra subslides.

Multiple implicit

Base content.

Phase B styled.

```
Base content.  
#uncover(<im-a>)[Phase A.]  
#effect(  
  text.with(fill: blue), <im-b>  
)[Phase B styled.]  
#only(<im-c>)[Phase C only.]
```

Each distinct label → one implicit waypoint.

Reusing a label adds no extra subslides.

Multiple implicit

Base content.

```
#uncover(<im-a>)[Phase A.]  
#effect(  
  text.with(fill: blue), <im-b>  
)[Phase B styled.]  
#only(<im-c>)[Phase C only.]
```

Base content.
C only.

Phase B styled. Phase

Each distinct label → one implicit waypoint.
Reusing a label adds no extra subslides.

Mixed explicit + implicit

```
#waypoint(<expl>)  
Explicit phase.  
#uncover(<impl>)[  
    Implicit phase.  
]  
#uncover(from-wp(<expl>))[  
    From explicit onward.  
]
```

Mixed explicit + implicit

```
#waypoint(<expl>)  
Explicit phase.  
#uncover(<impl>)[  
    Implicit phase.  
]  
#uncover(from-wp(<expl>))[  
    From explicit onward.  
]
```

Explicit phase.
onward.

From explicit

Mixed explicit + implicit

```
#waypoint(<expl>)  
Explicit phase.  
#uncover(<impl>)[  
    Implicit phase.  
]  
#uncover(from-wp(<expl>))[  
    From explicit onward.  
]
```

Explicit phase. Implicit phase. From explicit onward.

Forward References

Using waypoints before they are defined

Forward reference

Shown before summary. Content.

```
// Reference before definition
#uncover(until-wp(<summary>))[
  Shown before summary.
]
Content.
#waypoint(<summary>)
Summary text.
#uncover(from-wp(<summary>))[
  Summary visible.
]
```

Waypoints can be referenced **before** they are defined on the same slide. `from-wp/` `until-wp` are lazy markers resolved at render time — after all waypoints are collected.

Using waypoints before they are defined

Forward reference

```
// Reference before definition
#uncover(until-wp(<summary>))[
  Shown before summary.
]
Content.
#waypoint(<summary>)
Summary text.
#uncover(from-wp(<summary>))[
  Summary visible.
]
```

Content. Summary
text. Summary visible.

Waypoints can be referenced **before** they are defined on the same slide. `from-wp/` `until-wp` are lazy markers resolved at render time — after all waypoints are collected.

Rules for waypoint references

- **Forward references work.** `from-wp(<lbl>)`, `until-wp(<lbl>)`, and bare `<lbl>` can appear before `#waypoint(<lbl>)` (or other waypoint creating functions) on the same slide.
- **Cross-slide references do not work.** Waypoints are scoped to a single slide. A label defined on slide 3 cannot be used on slide 5.
- **Undefined waypoints are errors.** If a label is referenced but never defined on the slide (by `#waypoint` or an implicit label), Touying raises an error at the end of the slide.
- **Implicit waypoints are created at first use.** `#uncover(<lbl>)` both references and defines the waypoint — a forward reference is never needed for implicit waypoints.

Advanced Usage

#alternatives with at:

```
#waypoint(<opt-a>)  
Option A active.  
#waypoint(<opt-b>)  
Option B active.  
#alternatives(  
  at: (<opt-a>, <opt-b>)  
) [Content A.] [Content B.]
```

Maps each body to a named waypoint instead of sequential numbering.

#alternatives with at:

Option A active.

Content **A**.

```
#waypoint(<opt-a>)  
Option A active.  
#waypoint(<opt-b>)  
Option B active.  
#alternatives(  
  at: (<opt-a>, <opt-b>)  
) [Content A.] [Content B.]
```

Maps each body to a named waypoint instead of sequential numbering.

#alternatives with at:

Option A active. Option B active. Content **B**.

```
#waypoint(<opt-a>)  
Option A active.  
#waypoint(<opt-b>)  
Option B active.  
#alternatives(  
  at: (<opt-a>, <opt-b>)  
) [Content A.] [Content B.]
```

Maps each body to a named waypoint instead of sequential numbering.

item-by-item — relative (auto)

Text before.

```
Text before.  
#pause  
After first pause.  
#item-by-item[  
  - First item  
  - Second item  
  - Third item  
]  
#pause  
After second pause.
```

With `start: auto` (default), items continue from the current pause position.

They always stay for the remainder of the slide.

item-by-item — relative (auto)

Text before.

#pause

After first pause.

#item-by-item[

- First item
- Second item
- Third item

]

#pause

After second pause.

Text before. After first pause.

- First item

With **start: auto** (default), items continue from the current pause position.

They always stay for the remainder of the slide.

item-by-item — relative (auto)

```
Text before.  
#pause  
After first pause.  
#item-by-item[  
  - First item  
  - Second item  
  - Third item  
]  
#pause  
After second pause.
```

Text before. After first pause.

- First item
- Second item

With `start: auto` (default), items continue from the current pause position.

They always stay for the remainder of the slide.

item-by-item — relative (auto)

```
Text before.  
#pause  
After first pause.  
#item-by-item[  
  - First item  
  - Second item  
  - Third item  
]  
#pause  
After second pause.
```

Text before. After first pause.

- First item
- Second item
- Third item

With `start: auto` (default), items continue from the current pause position.

They always stay for the remainder of the slide.

item-by-item — relative (auto)

```
Text before.  
#pause  
After first pause.  
#item-by-item[  
  - First item  
  - Second item  
  - Third item  
]  
#pause  
After second pause.
```

Text before. After first pause.

- First item
- Second item
- Third item

After second pause.

With `start: auto` (default), items continue from the current pause position.

They always stay for the remainder of the slide.

item-by-item with waypoint start

```
#waypoint(<list-wp>)  
#item-by-item(start: <list-wp>)[  
  - Alpha  
  - Beta  
  - Gamma  
]  
#uncover(from-wp(<list-wp>))[  
  List revealed above.  
]  
#only(<final>)[  
  Finally after the list.  
]
```

Anchor the item reveal to a named waypoint
(implicit also possible).

item-by-item with waypoint start

- Alpha

```
#waypoint(<list-wp>)  
#item-by-item(start: <list-wp>)[  
  - Alpha  
  - Beta  
  - Gamma  
]  
#uncover(from-wp(<list-wp>))[  
  List revealed above.  
]  
#only(<final>)[  
  Finally after the list.  
]
```

List revealed above.

Anchor the item reveal to a named waypoint
(implicit also possible).

item-by-item with waypoint start

```
#waypoint(<list-wp>)  
#item-by-item(start: <list-wp>)[  
  - Alpha  
  - Beta  
  - Gamma  
]  
#uncover(from-wp(<list-wp>))[  
  List revealed above.  
]  
#only(<final>)[  
  Finally after the list.  
]
```

- Alpha
- Beta

List revealed above.

Anchor the item reveal to a named waypoint
(implicit also possible).

item-by-item with waypoint start

```
#waypoint(<list-wp>)  
#item-by-item(start: <list-wp>)[  
  - Alpha  
  - Beta  
  - Gamma  
]  
#uncover(from-wp(<list-wp>))[  
  List revealed above.  
]  
#only(<final>)[  
  Finally after the list.  
]
```

- Alpha
- Beta
- Gamma

List revealed above.

Anchor the item reveal to a named waypoint
(implicit also possible).

item-by-item with waypoint start

```
#waypoint(<list-wp>)  
#item-by-item(start: <list-wp>)[  
  - Alpha  
  - Beta  
  - Gamma  
]  
#uncover(from-wp(<list-wp>))[  
  List revealed above.  
]  
#only(<final>)[  
  Finally after the list.  
]
```

- Alpha
- Beta
- Gamma

List revealed above.

Finally after the list.

Anchor the item reveal to a named waypoint
(implicit also possible).

Hierarchical Waypoints

```
#alternatives(at: (<intro>, <more>))  
[Introduction.][More details.]  
  #waypoint(<intro:background>)  
  Some background during intro.  
  #waypoint(<intro:goal>)  
  The goal during intro.  
  #waypoint(<more:analysis>)  
  Analysis during more.  
  #waypoint(<more:results>)  
  Results during more.
```

You may even construct hierarchical waypoints, which we collect automatically by their shared top level.

We use ‘:’ as the separator of levels.

Hierarchical Waypoints

```
#alternatives(at: (<intro>, <more>))  
[Introduction.][More details.]  
  #waypoint(<intro:background>)  
  Some background during intro.  
  #waypoint(<intro:goal>)  
  The goal during intro.  
  #waypoint(<more:analysis>)  
  Analysis during more.  
  #waypoint(<more:results>)  
  Results during more.
```

You may even construct hierarchical waypoints, which we collect automatically by their shared top level.

We use ‘:’ as the separator of levels.

Introduction.

Some background during intro.

Hierarchical Waypoints

```
#alternatives(at: (<intro>, <more>))  
[Introduction.][More details.]  
  #waypoint(<intro:background>)  
  Some background during intro.  
  #waypoint(<intro:goal>)  
  The goal during intro.  
  #waypoint(<more:analysis>)  
  Analysis during more.  
  #waypoint(<more:results>)  
  Results during more.
```

You may even construct hierarchical waypoints, which we collect automatically by their shared top level.

We use ‘:’ as the separator of levels.

Introduction.

Some background during intro. The goal during intro.

Hierarchical Waypoints

```
#alternatives(at: (<intro>, <more>))  
[Introduction.][More details.]  
  #waypoint(<intro:background>)  
  Some background during intro.  
  #waypoint(<intro:goal>)  
  The goal during intro.  
  #waypoint(<more:analysis>)  
  Analysis during more.  
  #waypoint(<more:results>)  
  Results during more.
```

You may even construct hierarchical waypoints, which we collect automatically by their shared top level.

We use ‘:’ as the separator of levels.

More details.

Some background during intro. The goal during intro.

Analysis during more.

Hierarchical Waypoints

```
#alternatives(at: (<intro>, <more>))  
[Introduction.][More details.]  
  #waypoint(<intro:background>)  
  Some background during intro.  
  #waypoint(<intro:goal>)  
  The goal during intro.  
  #waypoint(<more:analysis>)  
  Analysis during more.  
  #waypoint(<more:results>)  
  Results during more.
```

You may even construct hierarchical waypoints, which we collect automatically by their shared top level.

We use ‘:’ as the separator of levels.

More details.

Some background during intro. The goal during intro.

Analysis during more. Results during more.

Navigating Hierarchies with `next-wp` / `prev-wp`

```
#waypoint(<before>, advance: false)
Before the group.
#waypoint(<grp:a>)
Part A.
#waypoint(<grp:b>)
Part B.
#waypoint(<after>)
After the group.
// Virtual parent (no <grp> waypoint):
// next-wp → last child + 1
#only(next-wp(<grp>))[Past the group (after).]
// prev-wp → first child - 1
#only(prev-wp(<grp>))[Before the group (prev)]
```

When `<grp>` is a *virtual* parent (only children like `<grp:a>` etc. exist), `next-wp` anchors to the **last** child and `prev-wp` to the **first** — so they navigate *past* or *before* the entire group.

Before the group.

Before the group (prev)

You can also reach children: `next-wp(get-first(<grp>))` starts at the first child and steps forward within the group.

If an explicit `#waypoint(<grp>)` exists, it anchors to that label directly — stepping into the children from there.

Navigating Hierarchies with `next-wp` / `prev-wp`

```
#waypoint(<before>, advance: false)
Before the group.
#waypoint(<grp:a>)
Part A.
#waypoint(<grp:b>)
Part B.
#waypoint(<after>)
After the group.
// Virtual parent (no <grp> waypoint):
// next-wp → last child + 1
#only(next-wp(<grp>))[Past the group (after).]
// prev-wp → first child - 1
#only(prev-wp(<grp>))[Before the group (prev)]
```

When `<grp>` is a *virtual* parent (only children like `<grp:a>` etc. exist), `next-wp` anchors to the **last** child and `prev-wp` to the **first** — so they navigate *past* or *before* the entire group.

Before the group. Part A.

You can also reach children: `next-wp(get-first(<grp>))` starts at the first child and steps forward within the group. If an explicit `#waypoint(<grp>)` exists, it anchors to that label directly — stepping into the children from there.

Navigating Hierarchies with `next-wp` / `prev-wp`

```
#waypoint(<before>, advance: false)
Before the group.
#waypoint(<grp:a>)
Part A.
#waypoint(<grp:b>)
Part B.
#waypoint(<after>)
After the group.
// Virtual parent (no <grp> waypoint):
// next-wp → last child + 1
#only(next-wp(<grp>))[Past the group (after).]
// prev-wp → first child - 1
#only(prev-wp(<grp>))[Before the group (prev)]
```

When `<grp>` is a *virtual* parent (only children like `<grp:a>` etc. exist), `next-wp` anchors to the **last** child and `prev-wp` to the **first** — so they navigate *past* or *before* the entire group.

Before the group. Part A. Part B.

You can also reach children: `next-wp(get-first(<grp>))` starts at the first child and steps forward within the group. If an explicit `#waypoint(<grp>)` exists, it anchors to that label directly — stepping into the children from there.

Navigating Hierarchies with `next-wp` / `prev-wp`

```
#waypoint(<before>, advance: false)
Before the group.
#waypoint(<grp:a>)
Part A.
#waypoint(<grp:b>)
Part B.
#waypoint(<after>)
After the group.
// Virtual parent (no <grp> waypoint):
// next-wp → last child + 1
#only(next-wp(<grp>))[Past the group (after).]
// prev-wp → first child - 1
#only(prev-wp(<grp>))[Before the group (prev)]
```

When `<grp>` is a *virtual* parent (only children like `<grp:a>` etc. exist), `next-wp` anchors to the **last** child and `prev-wp` to the **first** — so they navigate *past* or *before* the entire group.

Before the group. Part A. Part B. After the group.

Past the group (after).

You can also reach children: `next-wp(get-first(<grp>))` starts at the first child and steps forward within the group.

If an explicit `#waypoint(<grp>)` exists, it anchors to that label directly — stepping into the children from there.

Integration: CeTZ & Fletcher

Waypoints inside touying-reducer

Waypoints work inside touying-reducer, letting you name animation steps in CeTZ and Fletcher diagrams.

First, set up the reducer bindings (once, at the top of your file):

CeTZ:

```
#import "@preview/cetz:0.4.2"  
#let cetz-canvas = touying-  
reduce.with(cetz)
```

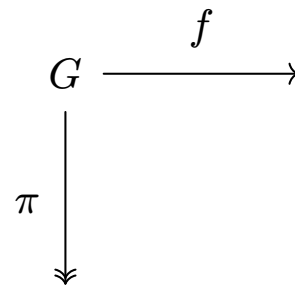
Fletcher:

```
#import "@preview/fletcher:0.5.8" as  
fletcher: edge, node  
#let fletcher-diagram = touying-  
reduce.with(fletcher)
```

Then use (waypoint(<lbl>),) for CeTZ or waypoint(<lbl>) for Fletcher inside the diagram — just like (pause,) or pause.

Fletcher: First Isomorphism Theorem

```
#fletcher-diagram(
  cell-size: 15mm,
  waypoint(<fl-maps>, advance: false),
  node((0, 0), $G$),
  edge((0, 0), (1, 0), $f$, "->"),
  edge((0, 0), (0, 1), $pi$, "->>"),
  pause,
  node((1, 0), $im(f)$),
  node((0, 1), $G\ker(f)$),
  waypoint(<fl-iso>),
  edge((0, 1), (1, 0), $tilde(f)$, "hook-->"),
)
#alternatives(
  at: (get-first(<fl-maps>), from-wp(get-last(<fl-maps>)))
)[
  $f: G -> $, $pi: G ->> $ ][
  $f: G -> im(f)$, $pi: G ->> G\ker(f)$ ]
#uncover(<fl-iso>[$tilde(f)$: the isomorphism.]
```



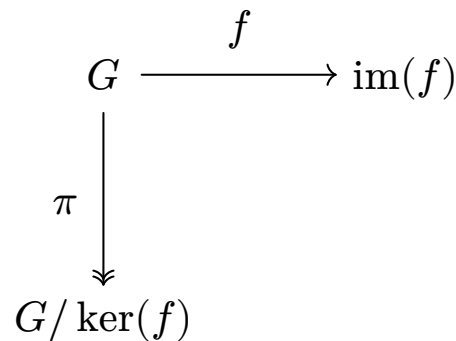
$$f : G \rightarrow,$$

$$\pi : G \twoheadrightarrow$$

Use math as labels, not for the whole diagram when using pause or waypoint inside it. We cannot recognize it otherwise.

Fletcher: First Isomorphism Theorem

```
#fletcher-diagram(
  cell-size: 15mm,
  waypoint(<fl-maps>, advance: false),
  node((0, 0), $G$),
  edge((0, 0), (1, 0), $f$, "->"),
  edge((0, 0), (0, 1), $pi$, "->>"),
  pause,
  node((1, 0), $im(f)$),
  node((0, 1), $G\ker(f)$),
  waypoint(<fl-iso>),
  edge((0, 1), (1, 0), $tilde(f)$, "hook-->"),
)
#alternatives(
  at: (get-first(<fl-maps>), from-wp(get-last(<fl-maps>)))
)[
  $f: G -> $, $pi: G ->> $ ][
  $f: G -> im(f)$, $pi: G ->> G\ker(f)$ ]
#uncover(<fl-iso>[$tilde(f)$: the isomorphism.]
```



$$f : G \rightarrow \text{im}(f),$$

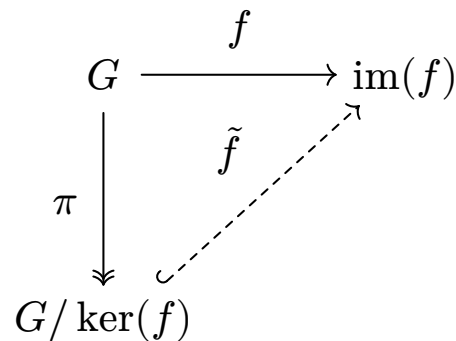
$$\pi : G \twoheadrightarrow G/\ker(f)$$

Use math as labels, not for the whole diagram when using pause or waypoint inside it. We cannot recognize it otherwise.

Fletcher: First Isomorphism Theorem

```
#fletcher-diagram(
  cell-size: 15mm,
  waypoint(<fl-maps>, advance: false),
  node((0, 0), $G$),
  edge((0, 0), (1, 0), $f$, "->"),
  edge((0, 0), (0, 1), $pi$, "->>"),
  pause,
  node((1, 0), $im(f)$),
  node((0, 1), $G\ker(f)$),
  waypoint(<fl-iso>),
  edge((0, 1), (1, 0), $tilde(f)$, "hook-->"),
)
#alternatives(
  at: (get-first(<fl-maps>), from-wp(get-last(<fl-maps>)))
)[
  $f: G -> $, $pi: G ->> $ ][
  $f: G -> im(f)$, $pi: G ->> G\ker(f)$ ]
#uncover(<fl-iso>[$tilde(f)$: the isomorphism.]
```

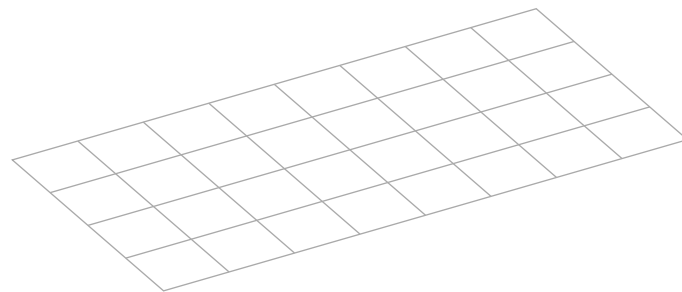
Use math as labels, not for the whole diagram when using pause or waypoint inside it. We cannot recognize it otherwise.



$f : G \rightarrow \text{im}(f)$,
 $\pi : G \twoheadrightarrow G/\ker(f)$
 $\tilde{f}$: the isomorphism.

CeTZ: 3D Sine Waves

```
#cetz-canvas({
  import cetz.draw: *
  let wave(amp, col) = { ... }
  (waypoint(<cz-grid>, advance: false),)
  ortho(y: -30deg, x: 30deg, {
    on-xz({grid((0,-2), (8,2), stroke: gray + .5pt)})
  })
  (waypoint(<cz-xy>),)
  ortho(y: -30deg, x: 30deg, {on-xy({ wave(1.6, blue) })})
  (waypoint(<cz-xz>),)
  ortho(y: -30deg, x: 30deg, {on-xz({ wave(1.0, red) })})
})
#only(<cz-grid>)[Grid plane.]
#only(<cz-xy>)[Blue wave (xy).]
#only(<cz-xz>)[Red wave (xz).]
```

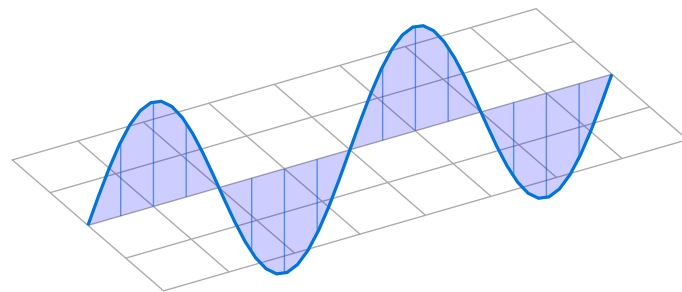


Grid plane drawn.

Similarly, for CeTZ, pause and waypoint must be at the top level of the canvas. Split `ortho(...)` and similar functions into separate calls — one per animation step. Do not put pause or waypoint inside functions like `ortho` or `on-xz`.

CeTZ: 3D Sine Waves

```
#cetz-canvas({  
  import cetz.draw: *  
  let wave(amp, col) = { ... }  
  (waypoint(<cz-grid>, advance: false),)  
  ortho(y: -30deg, x: 30deg, {  
    on-xz({grid((0,-2), (8,2), stroke: gray + .5pt)})  
  })  
  (waypoint(<cz-xy>),)  
  ortho(y: -30deg, x: 30deg, {on-xy({ wave(1.6, blue) })})  
  (waypoint(<cz-xz>),)  
  ortho(y: -30deg, x: 30deg, {on-xz({ wave(1.0, red) })})  
})  
#only(<cz-grid>)[Grid plane.]  
#only(<cz-xy>)[Blue wave (xy).]  
#only(<cz-xz>)[Red wave (xz).]
```

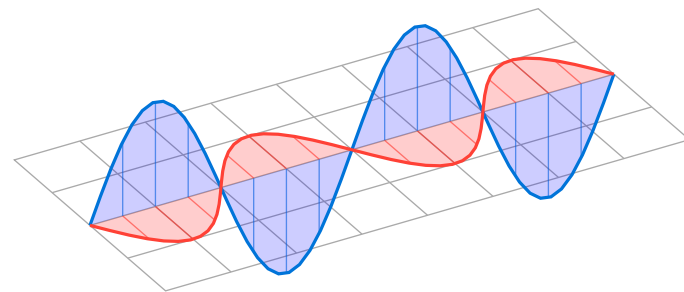


Blue wave on the xy-plane.

Similarly, for CeTZ, pause and waypoint must be at the top level of the canvas. Split `ortho(...)` and similar functions into separate calls — one per animation step. Do not put pause or waypoint inside functions like `ortho` or `on-xz`.

CeTZ: 3D Sine Waves

```
#cetz-canvas({
  import cetz.draw: *
  let wave(amp, col) = { ... }
  (waypoint(<cz-grid>, advance: false),)
  ortho(y: -30deg, x: 30deg, {
    on-xz({grid((0,-2), (8,2), stroke: gray + .5pt)})
  })
  (waypoint(<cz-xy>),)
  ortho(y: -30deg, x: 30deg, {on-xy({ wave(1.6, blue) })})
  (waypoint(<cz-xz>),)
  ortho(y: -30deg, x: 30deg, {on-xz({ wave(1.0, red) })})
})
#only(<cz-grid>)[Grid plane.]
#only(<cz-xy>)[Blue wave (xy).]
#only(<cz-xz>)[Red wave (xz).]
```



Red wave on the xz-plane.

Similarly, for CeTZ, pause and waypoint must be at the top level of the canvas. Split `ortho(...)` and similar functions into separate calls — one per animation step. Do not put pause or waypoint inside functions like `ortho` or `on-xz`.

Summary

Quick reference

Syntax	Effect
#waypoint(<lbl>)	Named #pause — marks + advances
#waypoint(<lbl>, advance: false)	Marks without advancing
#waypoint(<lbl>, start:int <lbl>)	Starts the waypoint at a specific subslide
#uncover(<lbl>)[...]	Show during waypoint range (implicit)
#only(<lbl>)[...]	Show only during range (implicit)
#effect(fn, <lbl>)[...]	Apply style during range (implicit)
get-first(<lbl>)	First subslide of the range
get-last(<lbl>)	Last subslide of the range
from-wp(<lbl>)	From waypoint to end of slide
until-wp(<lbl>)	Before waypoint (exclusive)
next-wp(<lbl>, amount:1)	Adjacent waypoint (forward), allows amount to skip multiple
prev-wp(<lbl>, amount:1)	Adjacent waypoint (backward), allows amount to skip multiple
(from-wp(<a>), until-wp())	Bounded range: <a> to before
not-wp(<lbl>)	Inverted range: not during <lbl>
#alternatives(at: (...))[...][...]	Named alternative mapping
#item-by-item[...]	Relative item reveal (auto from pause)

Summary

Syntax	Effect
<code>#item-by-item(start: <wp>)[...]</code>	Waypoint-anchored item reveal
<code><label:sublabel></code>	Hierarchical waypoint. The parent (e.g. <code><label></code>) refers to all its children (e.g. <code><label:sublabel></code>).
<code>(waypoint(<lbl>),)</code> inside CeTZ	Waypoint inside touying-reducer (CeTZ). Wrap in array like <code>(pause,)</code> .
<code>waypoint(<lbl>)</code> inside Fletcher	Waypoint inside touying-reducer (Fletcher). No tuple needed.